

Endless Domains Marketplace Frontend — Security, SEO & Performance Audit

Repo: endlessdomains-marketplace-frontend **Host:** marketplace.endlessdomains.io (subdomain — independent origin for security, independent site for crawlers) **Stack:** Next.js 15.1.4 (Pages Router), React 19, Redux Toolkit, wagmi / ethers / web3, Bootstrap 5 **Branch reviewed:** main @ 9e581ad **Date:** 2026-07-31 **Scope:** Non-design changes only.

Note on method: node_modules was not installed, so no production next build was run. Asset sizes and duplication counts are measured from files on disk; bundle-size findings are inferred from imports and dependency weight. Exact first-load JS figures need a real build to confirm.

Do these three first

#	Change	Why it can't wait
1	Rotate the Google OAuth secret; get .env out of git	A live client secret is in the repo and in every browser bundle
2	Fix the AppContent loading gate	Every page currently serves Loading... as its entire server HTML
3	Confirm production isn't calling apistage.endlessdomains.io	Production user data may be transiting a staging system

Security

S1 — Google OAuth client secret exposed in the browser bundle (CRITICAL)

Files: .env · next.config.ts:7 · pages/_app.tsx:23

.env contains GOOGLE_CLIENT_ID=GOCSPX-tDh... The GOCSPX- prefix is Google's format for a **client secret**, not a client ID — real client IDs end in .apps.googleusercontent.com. It has been mislabeled and put in the wrong variable.

next.config.ts then maps it into the env block as googleClientID. Next.js inlines env values into the client bundle at build time, and _app.tsx reads it and passes it to GoogleAuthProvider. **Anyone who opens devtools on production can read the secret.**

Fix 1. Rotate the secret in Google Cloud Console immediately. 2. Put the real client ID (...apps.googleusercontent.com) in NEXT_PUBLIC_GOOGLE_CLIENT_ID. 3. Keep the secret server-side only — it should never appear in next.config.ts's env block. 4. Verify Google sign-in actually works afterwards. Passing a secret where a client ID belongs suggests the flow may currently be broken.

S2 — .env is committed to git (CRITICAL)

Files: .env · .env.cmdrc · .gitignore

Both files are tracked (git ls-files confirms). .gitignore has no .env entry at all. Every value — including S1 — is in git history.

Note that scripts/fetch_env.sh already pulls the env file from S3 at deploy time, so the committed copy isn't even needed.

Fix

```
printf '\n# Environment\n.env\n.env.*\n!.env.example\n.env.cmdrc\n' >> .gitignore
git rm --cached .env .env.cmdrc
```

Then treat every value in both files as compromised and rotate. Commit a .env.example with keys but no values.

S3 — Hardcoded API key in source (CRITICAL)

Files: src/constants/BaseUrl.ts:5 · src/component/modals/DomainListingModal.tsx:28

```
export const API_KEY: string | undefined = 'b077ef14b737bc4294a322a0be8848b443c6963a58cc21ba6b5e07628246ce9b'
```

Plain string in source: in the bundle, in git, readable by anyone.

Fix — Proxy whatever service this authenticates against through your own backend so the key never reaches the browser. Rotate it.

S4 — Auth tokens in JavaScript-readable cookies (CRITICAL)

Files: src/core/services/cookies.service.ts · auth-component:115 · wallet-signup:91 · google-signup:32

```
export const setCookieValue = (key: string, value: string) => {
  setCookie(key, value) // no httpOnly, no secure, no sameSite, no expiry
}
```

Both the access token and the **long-lived refresh token** are set this way from three separate call sites. Any XSS — including from a third-party script (see S8) — walks away with a refresh token and a persistent session.

Fix (preferred) — Have the backend set `httpOnly; Secure; SameSite=Lax` cookies on the auth response. The frontend then stops touching tokens entirely.

Fix (interim, if backend change is slower)

```
setCookie(key, value, { secure: true, sameSite: 'strict', path: '/' })
```

plus a short expiry on the access token.

Subdomain note

Cookies scoped to `.endlessdomains.io` are shared across **every** subdomain. Currently `cookies-next` defaults to host-only (`marketplace.endlessdomains.io`), which is the safe behaviour — but it's accidental, not deliberate. **Do not** add `domain: '.endlessdomains.io'` to make login work across the apex site: that would let an XSS on any sibling subdomain steal marketplace refresh tokens. `SameSite` does not protect you here, since sibling subdomains count as same-site.

S5 — Cookie names are SHA-256 hashes

File: `src/core/enum/cookie.enum.ts`

```
ACCESS_TOKEN = '86b3901eea37a04e8547cd912225f548d2e0a92078887682ce831a433072f9d1'
REFRESH_TOKEN = '6c8a7d4aa21708a432174e4cb5c6cfaf0218f5f3e52f9a76a7d95d2aaade2c83'
```

Obfuscation, not security. Costs nothing to an attacker and makes debugging harder.

Fix — Rename to `access_token` / `refresh_token` as part of the S4 work.

S6 — Route protection is client-side only, and inconsistent (HIGH)

Files: `src/component/ProtectedRoute.tsx` · `pages/profile/`

`ProtectedRoute` gates on React state, so page HTML and its JS chunk are served to anyone; the redirect only fires after hydration.

Three pages do it properly with `getServerSideProps` cookie checks: - `pages/register/index.tsx:21` (OK) - `pages/profile/domains/index.tsx:124` (OK)

But `/profile/analytics`, `/profile/user`, `/profile/userProfile`, `/profile/watchlist` have **no** server-side check.

Two `getServerSideProps` exports are dead code — Next.js only runs it from files under `pages/`, but these are in component files: - `src/component/auth-component/index.tsx:282` (never runs) - `src/component/forgot-password/index.tsx:141` (never runs)

Assuming your API enforces real authorization, this is data-exposure and UX rather than full auth bypass — but the gap should close.

Fix — One `middleware.ts` at the repo root covering `/profile/*` replaces all of it with less code:

```
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
import { Cookie_Key } from '@core/enum/cookie.enum'

export function middleware(req: NextRequest) {
  if (!req.cookies.get(Cookie_Key.ACCESS_TOKEN)) {
    return NextResponse.redirect(new URL('/login', req.url))
  }
  return NextResponse.next()
}

export const config = { matcher: ['/profile/:path*'] }
```

Then delete the two dead exports.

S7 — Refresh-token flow can loop and can throw (HIGH)

File: `src/core/services/error.service.ts:38-52`

A 401 triggers a refresh, and the caller retries the original request. Problems: - No guard against the refresh call itself returning 401 → potential loop. - No check that `response.result` exists before reading `.accessToken` → confusing `TypeError` on a failed refresh.

Compounding: the `apiEndpoint` this refresh call uses is built from undefined env vars — see **P11**. The refresh URL is literally `undefined/undefined/undefined/...`, so **this flow has likely never worked**.

Fix — Add a single-retry flag, validate the response shape, and route failure to a clean logout. Fix the URL (P11) first or the rest is moot.

S8 — Third-party scripts without SRI or version pinning (HIGH)

File: `src/template/layout/index.tsx:114-119`

Source	Issue
<code>ajax.googleapis.com</code> (jQuery 3.7.1)	No <code>integrity</code>
<code>cdnjs.cloudflare.com</code> (Font Awesome 6.5.1)	No <code>integrity</code> , render-blocking
<code>cdn.jsdelivr.net/npm/chart.js</code>	No <code>integrity</code> , no version pin
<code>cdn.jsdelivr.net/npm/chartjs-plugin-datalabels@2</code>	No <code>integrity</code>

Each is a full-privilege script execution slot on your marketplace.

Fix — Install as npm dependencies so they're versioned and audited with everything else. If they must stay on CDN, add `integrity` + `crossorigin="anonymous"` and pin exact versions. Chart.js can likely be **deleted outright** — you use Recharts (see P6).

S9 — jQuery loaded twice; probably not needed at all

File: `src/template/layout/index.tsx:61` and `:126`

A `useEffect` injects a jQuery `<script>` tag, and a static `<script>` tag loads it again. It also assigns `window.jQuery` / `window.$` globally.

Fix — In a React 19 app, ask whether jQuery is needed at all. Removing it eliminates the duplicate download, the global assignment, and the SRI concern in one move. See also P10 (the `setTimeout` DOM wiring it enables).

S10 — No security headers (HIGH)

File: `next.config.ts` — no `headers()` function at all.

Missing: CSP, `X-Frame-Options` / `frame-ancestors`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`.

Clickjacking matters here specifically: users sign wallet transactions on this site, and a lookalike subdomain framing your buy flow would be convincing.

Fix — ship immediately (5 lines, zero risk):

```
async headers() {
  return [{
    source: '/*:path*',
    headers: [
      { key: 'X-Content-Type-Options', value: 'nosniff' },
      { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
      { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
      { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
    ],
  }]
}
```

Fix — **CSP (more work, needs testing):** Prefer `frame-ancestors 'self'` over a blanket `DENY` if the apex site may ever embed marketplace widgets. Budget time for the CDN scripts and wallet SDKs — start in `Content-Security-Policy-Report-Only`.

S11 — ESLint disabled at build time

File: `next.config.ts:24-26`

```
eslint: { ignoreDuringBuilds: true }
```

Reinforced by `/* eslint-disable */` at the top of `api.service.ts`, `error.service.ts`, and `redux/api.ts`.

This is why several items on this list went uncaught.

Fix — Re-enable. It'll be noisy at first; it's the cheapest ongoing defense you have.

S12 — External links missing `rel="noopener noreferrer"`

Files: `src/component/footer/index.tsx:54-83` · `src/component/header/index.tsx:32`

~10 `target="_blank"` links. Modern browsers imply `noopener`, so severity is low — but it's a one-line sweep and also affects SEO link handling.

S13 — 48 `console.*` calls in production code

Several sit in error paths and print full API error bodies to the browser console.

Fix — Don't do it by hand:

```
compiler: { removeConsole: process.env.NODE_ENV === 'production' ? { exclude: ['error'] } : false }
```

Clean: XSS surface

Grepped for `dangerouslySetInnerHTML`, `innerHTML`, `eval()`, `document.write` across `pages/` and `src/` — **zero matches**. Your own code introduces no XSS sinks. This is genuinely good, and it's why S4 is a fix-the-storage problem rather than an active-breach one.

SEO

E1 — Every page serves `<p>Loading...</p>` to crawlers (CRITICAL)

Highest-impact item in this audit. Roughly five lines of code.

File: `src/component/AppContent.tsx:7`

```
const { authenticated } = useAuth();
if (authenticated === null) return <p>Loading...</p>;
```

`authenticated` starts as `null` and is only set inside a `useEffect` in `AuthContext` — which never runs on the server. So on **every** server render, for **every** page, the entire page tree (including each page's `<Head>` tags) is replaced by `<p>Loading...</p>`. No title, no description, no content in served HTML.

Three independent layers each blank the server HTML: 1. `AppContent.tsx:7` — `authenticated === null` → `Loading...` 2. `ProtectedRoute.tsx:32` — `loading` → `Loading...` 3. `pages/index.tsx` — whole body wrapped in `{isClient ? ... : ""}`

Google does execute JavaScript, so you aren't fully invisible — but rendered-HTML indexing is slower, less reliable, and strictly worse for ranking than served HTML. Every *other* crawler (Bing, social preview bots, AI crawlers) sees nothing but "Loading...".

Fix - Read the auth cookie during SSR so `authenticated` has a real value on first render. - Never gate the page subtree behind a client-only flag. Render the page; let auth-dependent *widgets* handle their own loading state. - Remove the `isClient` wrapper in `index.tsx`.

This is the same root cause found in the `webui` project — worth killing the pattern across both codebases.

E2 — No `robots.txt`, no sitemap (CRITICAL)

Neither exists in `public/`. Crawlers have no discovery path for domain listings or seller portfolios.

Subdomain note: a `robots.txt` on the apex does **not** cover this subdomain. Each host serves its own.

Fix - Add `public/robots.txt` pointing at `https://marketplace.endlessdomains.io/sitemap.xml`. - Generate a sitemap (dynamic route or `next-sitemap`) including `/seller-portfolio/[slug]` and domain detail pages — those are the money pages for organic search. - Register `marketplace.endlessdomains.io` as its own Search Console property (or use a domain property covering all subdomains). Without it you're blind on exactly the pages that matter.

E3 — No `_document.tsx`

Without it you can't set `<html lang="en">` — both an SEO signal and an accessibility requirement. It's also the right home for `preconnect` hints.

Fix — Add `pages/_document.tsx` with `lang="en"` and preconnects for any remaining third-party origins.

E4 — No canonical, Open Graph, Twitter cards, or structured data (CRITICAL)

Grepped for `canonical`, `og:title`, `og:image`, `twitter:card`, `application/ld+json` — **zero matches anywhere**.

Concretely: - Sharing a listing on X or Discord produces a bare link with no preview. - No defense against duplicate-content issues between staging and production hosts (see E9). - No rich results at all.

Subdomain note: Google treats `marketplace.endlessdomains.io` and `endlessdomains.io` as **separate sites with separate authority**. Every canonical, OG `url`, and sitemap entry must use the full `marketplace.` host. Pointing canonicals at the apex would actively deindex the marketplace.

Fix — Extend `MetaData` and the `<Head>` block in `layout` to emit all of it in one place: - `<link rel="canonical">` with the absolute `https://marketplace.endlessdomains.io/...` URL - Full OG set (`og:title`, `og:description`, `og:image`, `og:url`, `og:type`, `og:site_name`) - `twitter:card = summary_large_image` - JSON-LD: `Product` + `Offer` on listing pages, `BreadcrumbList` on portfolios, `Organization` + `WebSite` sitewide

E5 — No `<h1>` on any page

Page	<h1>	<h2>
pages/index.tsx	0	5
pages/about-us/index.tsx	0	4
pages/contact-us/index.tsx	0	2

Every page should have exactly one `<h1>`. **Pure content fix, no design change** — the heading text already exists, it's just at the wrong level.

E6 — `<meta name="keywords">` still emitted

File: `src/template/layout/index.tsx:102`

Google has ignored this since 2009. It mainly hands competitors your keyword list. Harmless but worth deleting — along with the `keywords` field on `MetaData`.

E7 — Dynamic pages have no per-page metadata

File: `pages/seller-portfolio/[slug].tsx`

Reads its slug from `router.query`, which is client-only, so the page can't produce a meaningful title or description server-side. Combined with E1, these pages are effectively unindexable.

Fix — Move to `getServerSideProps` (or `getStaticProps` + ISR) so you get real titles like *"Domains for sale by X"* and shareable previews.

E8 — Missing `favicon.ico` and unwired manifest

- `layout:106` links `/favicon.ico` — **no such file** in `public/`. Only `assets/img/Favicon.svg` exists. Every page load 404s.
- `MetaData` has a `manifest` field and `index.tsx:32` sets it to `site.webmanifest`, but the layout never renders a manifest link and **no `site.webmanifest` file exists**.
- `next-pwa` is in `dependencies` but there's no PWA config in `next.config.ts` — currently dead weight.

Fix — Add the favicon, then either wire up the manifest + `next-pwa` properly or drop both.

E9 — Check for a crawlable staging clone (HIGH)

`stagemarketplace.endlessdomains.io` is referenced twice in the code. If that host is publicly reachable and crawlable, it's duplicate content competing against production — and Google sometimes picks the wrong one to index. With no `robots.txt` on any host, nothing currently prevents this.

Fix — `curl` the host, run a `site:endlessdomains.io` search. If it's public, add `X-Robots-Tag: noindex` or HTTP auth.

E10 — Cross-linking with the apex

Subdomains don't inherit apex authority the way subdirectories do. The footer's `target="_blank"` links to `endlessdomains.io/blog`, `/community-page`, `/vision-mission` push equity **out** to the apex.

Fix — Get apex blog content linking *into* marketplace listing pages. Costs nothing, real ranking gains.

Performance

P1 — Bootstrap CSS loaded three times per page (CRITICAL)

Location	What
<code>_app.tsx:20</code>	<code>import '../public/assets/css/bootstrap.min.css'</code>
<code>_app.tsx:21</code>	<code>import 'bootstrap/dist/css/bootstrap.min.css'</code> (second copy of the same framework)
<code>layout:108</code>	<code><link rel="stylesheet" href="/assets/css/bootstrap.min.css" /></code> (third)

~470 KB of CSS where 156 KB would do. Delete two of the three.

P2 — Every CSS file linked twice (CRITICAL)

File: `src/template/layout/index.tsx:108-121`

The layout `<link>`s `custom.css`, `style.css`, `responsive.css`, and both owl-carousel files inside `<Head>` — but `_app.tsx` already imports three of them

as modules. The two owl files are linked **a second time** at lines 120-121, *outside* `<Head>` entirely, which is invalid placement.

~300 KB of duplicate CSS on every page.

Fix — Pick one mechanism (the `_app.tsx` imports) and delete every `<link rel="stylesheet">` from the layout.

File	Size
<code>bootstrap.min.css</code>	156 KB
<code>custom.css</code>	72 KB
<code>style.css</code>	44 KB
<code>responsive.css</code>	16 KB
<code>owl.carousel.min.css</code>	4 KB
<code>owl.theme.default.min.css</code>	4 KB
Total	296 KB (x2-3 as currently loaded)

P3 — 22 MB of fonts in `public/` (CRITICAL)

Directory: `public/assets/fonts/`

All 18 Lato weights, each in three formats (`.ttf` , `.woff` , `.woff2`). The 18 `.ttf` files are pure dead weight — every browser you support handles `woff2` .

Fix — Realistically you use 2-3 weights. Dropping to woff2-only for those takes this from **22 MB to under 200 KB**. Better still: `next/font` self-hosts, subsets automatically, and eliminates font-swap layout shift.

P4 — A 1.3 MB PDF in the images directory (CRITICAL)

`public/assets/img/male1 (1).pdf` — 1.3 MB, referenced nowhere. Delete it.

Oversized SVGs (almost certainly embedded raster data in an SVG wrapper):

File	Size
<code>thankyou.svg</code>	1.3 MB
<code>promot-bg.svg</code>	1.3 MB
<code>domain-bg.svg</code>	1.3 MB
<code>f-graphic-a.svg</code>	644 KB
<code>f-graphic-b.svg</code>	636 KB

Fix — Run through SVGO, or re-export properly. Expect 90%+ reduction.

P5 — 106 raw `<img>` tags vs 7 files using `next/image` (CRITICAL)

No lazy loading, no automatic WebP/AVIF, no `width` / `height` (so every one contributes to CLS), no responsive `srcset` .

With `why-banner.png` at 544 KB, `why-domain.png` at 400 KB, `arch.png` at 356 KB, `about-us.png` at 340 KB — **this is likely your single biggest LCP problem.**

Fix — Start with above-the-fold homepage images. Converting just those moves the needle measurably.

P6 — Only one `next/dynamic` call in the whole app (HIGH)

Everything below is in the initial bundle:

Library	Files referencing	Notes
<code>web3</code> (v1)	12	One of the heaviest packages in the ecosystem
<code>react-bootstrap</code>	23	
<code>ethers</code> (v6)	4	
<code>wagmi</code>	4	
<code>@reown/appkit</code>	2	Plus both Solana and Wagmi adapters
<code>recharts</code>	3	Only needed on the analytics page

Fix — Three dynamic-import groups, biggest wins first: 1. **Wallet SDKs** (`@reown/appkit` , `web3` , `wagmi` connectors) — nobody needs these parsed before clicking "connect wallet" 2. **Recharts** — analytics page only 3. **Modals** — the 20+ files in `src/component/modals/` are all conditionally rendered

Redundant dependencies

You have **three** libraries doing the same job — `web3` v1, `ethers` v6, and `wagmi`. Consolidating on `ethers` + `wagmi` removes a large dependency outright.

Also likely leftovers: - `@walletconnect/web3-provider` — v1, deprecated - `@web3modal/wagmi` — superseded by `@reown/appkit`, which you also have - `next-pwa` — installed but unconfigured (see E8)

P7 — Bootstrap JS loaded twice, one via runtime `require`

Files: `_app.tsx:27` · `layout:129`

```
useEffect(() => { require('bootstrap/dist/js/bootstrap.bundle.min.js') }, [])
```

...and the layout *also* loads `/assets/js/bootstrap.bundle.min.js` via a script tag.

Duplicate download, and the `require` form defeats tree-shaking entirely.

P8 — Render-blocking third-party CSS

File: `src/template/layout/index.tsx:114`

Font Awesome from cdnjs blocks first paint on a cross-origin request with no `preconnect`.

Fix — Self-host, or subset to the handful of icons actually used. Removes a blocking round-trip.

P9 — Raw `<script>` tags instead of `next/script`

File: `src/template/layout/index.tsx:118-130`

Chart.js, chartjs-datalabels, jQuery, and two local scripts are all plain `<script>` tags in the layout body. The Facebook Pixel at `pages/reset-password/success/index.tsx:19` is another candidate.

Fix — `next/script` with `strategy="lazyOnload"` for non-critical scripts. **Delete Chart.js entirely** — you use Recharts.

P10 — Hardcoded 1-second `setTimeout` for DOM wiring

File: `src/template/layout/index.tsx:79`

```
const timerId = setTimeout(() => {
  const button = document.getElementById("left-menu-toggle");
  if (button) button.addEventListener("click", handleToggle);
}, 1000);
```

Both a race condition (slow devices may not have the element yet) and a guaranteed one-second dead zone for that control.

Fix — React ref + `onClick`. Removes the `getElementById`, the timeout, and the cleanup.

P11 — API base URL hardcoded to staging; other endpoints resolve to `undefined` (CRITICAL)

Hardcoded staging host

Files: `src/constants/BaseUrl.ts:4` · `src/core/services/api.service.ts:92`

```
export const baseUrl = `https://apistage.endlessdomains.io/api/v1` // env version commented out above
...
const response = await axios.post(`https://apistage.endlessdomains.io/api${url}`, formData)
```

If this is what's deployed to production, **every user's requests hit your staging API**. Correctness, latency, and data-handling problem.

Subdomain note: `marketplace.` → `apistage.` is a cross-origin call requiring CORS on the staging API — meaning staging must accept production origins. Audit that CORS config on the backend.

Undefined env vars breaking a second set of endpoints

Files: `api.service.ts:8-9` · `error.service.ts:5`

```
const apiEndpoint = `${process.env.apiDomainUrl}/${process.env.apiUrl}/${process.env.apiVersion}`
```

None of `apiDomainUrl`, `apiUrl`, `apiVersion`, `version`, `apiType`, `resellerID`, `udDomainUrl` are declared in the `env` block of `next.config.ts`. They all resolve to `undefined`, producing URLs like `undefined/undefined/undefined/...`

Affected: `putApi`, `putApiCall`, `deleteApi`, `deleteBulkApi`, `getCryptoAddress`, and **the token refresh call** (which is why S7 has likely never worked).

Fix — Single source of truth for API config, driven by `NEXT_PUBLIC_API_URL` per environment. Remove both hardcoded staging URLs and every `process.env.<undeclared>` reference.

Summary

By severity

CRITICAL (8) — S1 Google secret · S2 committed `.env` · S3 hardcoded API key · S4 token cookies · E1 SSR loading gate · E2 no robots/sitemap · E4 no canonical/OG/schema · P11 staging URL + undefined endpoints

HIGH (7) — S6 client-only route guards · S7 refresh loop · S8 no SRI · S10 no security headers · E9 staging clone · P1-P5 asset weight · P6 no code splitting

MEDIUM / LOW (rest) — S5, S9, S11, S12, S13, E3, E5, E6, E7, E8, E10, P7, P8, P9, P10

Cheapest wins by effort-to-impact

Change	Effort	Impact
Delete duplicate CSS <code><link></code> s (P1, P2)	~10 line deletion	~300 KB/page
Delete <code>.ttf</code> fonts + stray PDF (P3, P4)	<code>rm</code>	~20 MB repo, faster deploys
Add basic security headers (S10)	5 lines	Clickjacking + MIME sniffing
Add <code><h1></code> per page (E5)	Tag rename	Real ranking signal
Delete Chart.js (P9)	2 line deletion	One fewer CDN dependency
Fix <code>AppContent</code> gate (E1)	~5 lines	All SEO unblocked

Open items to verify

1. **Is production really calling `apistage.endlessdomains.io` ?** (P11) — highest-urgency unknown.
2. **Is `stagemarketplace.endlessdomains.io` publicly crawlable?** (E9)
3. **Does Google sign-in currently work?** (S1) — the mislabeled secret suggests it may not.
4. **Exact bundle sizes** — needs `npm install && npm run build`. Asset sizes above are measured from disk; JS bundle findings are inferred from imports.